Innovation Centre for Education

NCS
NihonCommunicationSolutions

YENEPOYA
(DEEMED TO BE UNIVERSITY)

YENEPOYA

(Deemed To Be University)

-

Final Project Report

on

Cloud Cost Intelligence Platform for Smart City

Student Name:

Malavika Rajesh

Industry Mentor: Mr. Shashank

Guided By: Mrs. Niharika

Table of Contents

1. EXECUTIVE SUMMARY 3
2. BACKGROUND 4
2.1 AIM 4
2.2 TECHNOLOGIES 5
2.3 HARDWARE ARCHITECTURE 6
2.4 SOFTWARE ARCHITECTURE 6
3. SYSTEM 8
3.1 REQUIREMENTS 9
3.1.1 Functional Requirements ..... *S*
3.1.2 User Requirements *10*
3.1.3 Environmental Requirements *11*
3.2 DESIGN AND ARCHITECTURE 11
3.3 IMPLEMENTATION 25
3.4 TESTING 27
3.5 GRAPHICAL USER INTERFACE (GUI) LAYOUT 29
3.6 CUSTOMER TESTING 30
3.7 EVALUATION 31
4. SNAPSHOTS OF THE PROJECT 33
5. CONCLUSION 40
6. FURTHER DEVELOPMENT OR RESEARCH 41
7. REFERENCES 42
8. APPENDIX 43
G.WEEKLY REPORT 44
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 01 44
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 02 46
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 03 48
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 04 50
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 05 52
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 06 54
WEEKLY PROJECT PROGRESS REPORT (WPPR)– 07 56

1.Executive Summary

This project presents the design and development of a Cloud Cost Intelligence Platform for Smart City, aimed at analyzing and visualizing cloud billing data in a simple and effective way. The system is developed as a web-based dashboard that allows users to upload cloud cost data in CSV format and instantly generate meaningful insights. It focuses on transforming raw data into clear visual information to support better cost management and decision-making.

The platform is built using Flask (Python) for backend processing, along with Pandas for data analysis and Matplotlib for visualization. The system follows a structured workflow where the uploaded data is validated, processed, and aggregated based on parameters such as Service Type, Department, Region, and Date. The results are then displayed through an interactive dashboard using Jinja2 templates, ensuring smooth integration between frontend and backend.

The application includes key features such as cost aggregation, trend analysis, and graphical representation through pie charts, bar graphs, and line charts. It also provides important insights like total cost, average daily cost, top department, and top application usage.

Testing was performed using different CSV datasets to verify data validation, accurate calculations, and correct chart generation. The results confirm that the system functions reliably and produces meaningful visual insights.

Overall, the project successfully develops a Cloud Cost Intelligence Platform for Smart City that analyzes and visualizes cloud billing data in a simple and effective way.

2. Background

The rapid growth of cloud computing in smart city environments has created a strong need for efficient systems to monitor and control cloud expenses. Cloud platforms generate large volumes of billing data across multiple dimensions such as services, departments, regions, and applications. However, understanding this data manually is complex, and many existing academic projects lack practical implementation with clear and interactive visualization.

To address this challenge, this project introduces a Cloud Cost Intelligence Platform that transforms raw cloud billing data into meaningful and easy-to-understand insights. The system enables users to upload CSV files containing cloud cost data and instantly view analyzed results through a web-based dashboard. It provides a clear breakdown of costs, helping users identify spending patterns and optimize resource usage.

The platform is designed using a Flask-based architecture that follows a simple and efficient request-response workflow. When a user uploads a CSV file, the system processes the data using Pandas, performs validations and aggregations, and generates visual outputs using Matplotlib. These outputs are then displayed as an interactive dashboard using Jinja2 templates, ensuring a seamless integration between data processing and user interface.

From a design and implementation perspective, the system follows a lightweight and modular approach. All data processing is performed in memory without permanent storage, ensuring faster execution and simplicity.

The project demonstrates how cloud billing data can be converted into valuable insights through structured data analysis, visualization, and system design, making it highly relevant for smart city cloud management.

2.1 Aim

The primary aim of this project is to design and develop a Cloud Cost Intelligence Platform for Smart City that provides a simple, efficient, and interactive way to analyze cloud usage and expenses.

This platform transforms raw cloud billing data into meaningful insights by allowing users to upload CSV files and visualize cost patterns through a web-based dashboard. It integrates data processing, analysis, and visualization into a single workflow, making complex cloud cost data easy to understand and manage.

Specific aims include:

• To develop a web-based platform for uploading and analyzing cloud billing CSV data.

• To process and validate data using Pandas for accurate analysis.

• To group and analyze costs based on Service Type, Department, Region, and Date.

• To generate visual insights such as pie charts, bar graphs, and trend lines.

• To display key performance indicators (KPIs) like total cost and average daily cost.

• To design a simple and interactive dashboard using Flask and Jinja2.

• To ensure a lightweight system with in-memory processing and no permanent data storage.

• To demonstrate practical data analytics skills applied to cloud and smart city management.

2.2 Technologies

The project uses simple and effective technologies for data analysis and web-based visualization:

• Python – Used for overall application development and data processing.

• Flask – Web framework used to build the backend and handle file uploads and routing.

• Pandas – Used for data analysis, cleaning, and grouping of cloud billing data.

• Matplotlib – Used to generate charts such as pie charts, bar graphs, and trend lines.

• HTML5 & CSS3 – Used to design the structure and styling of the web dashboard.

• Jinja2 – Template engine used to connect backend data with frontend display.

• CSV (Comma-Separated Values) – Used as the input format for cloud billing data.

• Jupyter Notebook (optional) – Used during development for testing and data analysis.

2.3 Hardware Architecture

The current implementation follows a single-machine development architecture, making it simple and suitable for academic demonstration and testing.

The system is divided into three main components:

• **Client Side:** A web browser (such as Chrome, Edge, or Firefox) is used to access the application. It renders the dashboard interface and displays charts and cost insights generated by the system.

• **Application Server:** The Flask-based application runs locally on the system. It handles CSV file uploads, processes data using Pandas, generates visualizations using Matplotlib, and returns the results as a fully rendered web page.

• **Storage Unit:** The system does not use permanent storage. All uploaded CSV data is processed temporarily in-memory during the request and discarded after generating the output. This ensures a stateless and efficient system.

Minimum recommended hardware:
• Processor: Dual-core or above
• RAM: 4 GB minimum (8 GB recommended)
• Storage: 1 GB free space (project size is minimal)
• Network: Not required for local deployment, but required for future cloud hosting

This architecture is intentionally lightweight and compact, allowing easy debugging and demonstrating the complete data processing workflow.

2.4 Software Architecture

The software architecture is organized into three primary layers:

1. Presentation Layer
• HTML/CSS (Jinja2 templates)
This layer is responsible for rendering the dashboard interface, displaying charts, KPIs, and handling

user interactions such as CSV file upload. It provides visual feedback to the user through a clean and interactive web page.

2. Application / Business Layer

• app.py

This layer contains the core logic of the system. It handles file uploads, validates CSV data, processes it using Pandas, performs aggregations, and generates visualizations using Matplotlib. It also manages request handling and prepares data to be displayed on the frontend.

3. Data Processing Layer

• Pandas Data Frame (in-memory)

This layer handles data ingestion, cleaning, transformation, and analysis. The uploaded CSV file is converted into a Data Frame, where operations like grouping, filtering, and trend analysis are performed. No permanent storage is used.

Architectural Characteristics:

• In-memory data processing without using any database or permanent storage

• Lightweight single-machine architecture suitable for academic demonstration

• Modular design with separate functions for data processing, validation, and visualization

• Clear separation of concerns between presentation, business logic, and data processing layers

• Request–response-based workflow for CSV upload and dashboard rendering

• Efficient data handling using Pandas for fast aggregation and analysis

• Dynamic chart generation using Matplotlib with real-time data visualization

• Server-side rendering using Flask and Jinja2 templates

• Base64 encoding used to embed chart images directly into HTML (no external files needed)

3.System

The system is a web-based Cloud Cost Intelligence Platform for Smart City designed to analyze and visualize cloud billing data in an efficient and user-friendly manner. The primary purpose of the system is to transform raw cloud usage data into meaningful insights that help users understand cost distribution, usage trends, and resource consumption patterns. By providing clear visual outputs, the system supports better decision-making and cost optimization in cloud environments.

The platform operates by accepting a CSV file containing cloud billing data as input. This data typically includes attributes such as *Date, Cost, Department, Service Type, Application,* and *Region*. Once the file is uploaded through the web interface, the system processes the data and generates analytical results. These results are then displayed on a dashboard in the form of graphs and key performance indicators (KPIs), making complex data easy to interpret.

From a technical perspective, the system follows a client-server architecture. The frontend is developed using HTML, CSS, and Jinja2 templates, which are responsible for rendering the user interface and displaying the results. The backend is implemented using the Flask framework in Python, which handles request processing, file uploads, and data analysis. Communication between the frontend and backend occurs through standard HTTP request-response mechanisms.

The core functionality of the system lies in its data processing workflow. When a user uploads a CSV file, the Flask server receives the file and passes it to the processing module. The data is then read into a Pandas Data Frame, where it undergoes several steps including validation, cleaning, and transformation. Invalid or missing values are handled appropriately to ensure accurate analysis. The system then performs grouping and aggregation operations based on different parameters such as service type, department, and region. It also calculates metrics like total cost and average daily cost.

Once the data is processed, the system generates visual representations using Matplotlib. These include pie charts for service-wise cost distribution, bar graphs for department and regional analysis, and line charts for cost trends over time. These visualizations provide a clear and intuitive understanding of how

cloud resources are utilized and where costs are concentrated. The generated charts are encoded and embedded into the web page, allowing them to be displayed directly in the browser.

The system is designed to be lightweight and stateless, meaning that it does not store any data permanently. All uploaded data is processed temporarily in memory and discarded after the response is generated. This approach simplifies the architecture, improves performance, and reduces storage requirements. It also ensures that each request is independent, making the system more reliable and easier to maintain.

Overall, the system demonstrates a complete end-to-end workflow that includes data input, processing, analysis, and visualization. It integrates multiple technologies such as Flask, Pandas, and Matplotlib to create a practical and effective solution for cloud cost analysis. The platform serves as a strong foundation for future enhancements such as real-time data integration, interactive dashboards, and advanced analytics features, making it highly relevant for smart city cloud management applications.

3.1 Requirements

The requirements of this project were defined to ensure that the system is functionally complete, efficient, user-friendly, and capable of analyzing cloud cost data accurately in a smart city context.

3.1.1 Functional Requirements

The system must:

• Allow users to upload cloud billing data in CSV format.

• Validate the CSV file structure and check for required columns (Date, Cost_INR, Service_Type, Department, Application, Region).

• Handle missing or incorrect data values safely.

• Convert uploaded data into a Pandas DataFrame for processing.

• Clean and preprocess data by removing invalid or null entries.

• Perform cost aggregation based on Service Type, Department, Region, and Date.

• Calculate key metrics such as total cost, average daily cost, and top cost contributors.

• Generate multiple visualizations including:

Pie chart for service distribution

Bar charts for department and region analysis

Line chart for cost trends over time

• Convert charts into displayable format (base64 images).

• Render a complete dashboard with KPIs and visual charts.

• Support single request-response workflow for data processing and display.

• Provide clear error messages for invalid inputs or processing failures.

• Ensure fast processing even for moderate-sized datasets.

3.1.2 User Requirements

The user expects:

• A simple and intuitive web interface.

• Easy file upload functionality without complex steps.

• Quick visualization of cloud cost insights.

• Clear and readable charts for better understanding.

• Immediate feedback after uploading data.

• Informative error messages when issues occur.

• Smooth navigation and minimal interaction effort.

• Reliable and consistent system performance.

• No requirement for technical expertise to use the system.

3.1.3 Environmental Requirements

Execution environment includes:

• OS: Windows / Linux / macOS

• Python 3.x installed

• Required libraries: Flask, Pandas, Matplotlib

• Web browser (Chrome, Edge, Firefox) with JavaScript support

• Minimum hardware: 4 GB RAM, dual-core processor

• Localhost environment (127.0.0.1:5000) for running the application

• No database or external server required

• Internet not required for local execution (optional for deployment)

3.2 Design and Architecture

The system follows a client-server architecture with a simple and efficient request-response workflow for processing cloud billing data.

• User uploads a CSV file through the web interface.

• The browser sends an HTTP POST request to the Flask server.

- The Flask application receives the file and validates the input data.
- The system processes the CSV using Pandas for cleaning and aggregation.
- Charts and key metrics are generated using Matplotlib.
- Processed results are embedded into the HTML template.
- The response is sent back to the browser.
- The dashboard is rendered with updated charts and insights.

Key Design Decisions:

- Keep backend data processing centralized using Flask and Pandas.
- Avoid direct manipulation of data on the frontend.
- Perform all data validation and analysis on the server side.
- Use in-memory processing instead of database storage for simplicity.
- Generate charts dynamically for real-time visualization.
- Maintain modular functions for data processing and chart generation.
- Ensure clear separation between UI (frontend) and logic (backend).
- Keep the system lightweight and easy to debug.

Purpose:

Updates the file name label dynamically when a user selects a CSV file using the file input

```
<script>
function updateName(input) {
    const el = document.getElementById('fname');
    if (input.files[0]) {
        el.textContent = input.files[0].name;
        el.classList.remove('empty');
    } else {
        el.textContent = 'no file chosen';
        el.classList.add('empty');
    }
}
</script>
```

Observed result:

The selected file name is displayed instantly, and the default “no file chosen” message appears when no file is selected.

Purpose:

Handles CSV file upload, processes data using `analyze_costs()`, generates key performance indicators (KPIs), creates charts, and sends the results to web dashboard.

```
if request.method == "POST":
    file = request.files.get("myfile")
    filename = file.filename if file else None

    if file and file.filename != "":
        try:
            analysis = analyze_costs(file)
            kpis = {
                "total_cost":           f"{analysis['total_cost']:,}",
                "avg_daily_cost":        f"{analysis['avg_daily_cost']:,}",
                "top_department_name":   analysis['top_department_name'],
                "top_department_cost":   f"{analysis['top_department_cost']:,}",
                "top_application_name":  analysis['top_application_name'],
                "top_application_cost":  f"{analysis['top_application_cost']:,}",
            }
            charts = {
                "service_pie":      create_chart_image(plot_service_pie(analysis['service_costs'])),
                "department_bar":   create_chart_image(plot_bar(analysis['department_cost'], 'Cost by Department', 'Department')),
                "region_bar":       create_chart_image(plot_bar(analysis['region_cost'], 'Cost by Region', 'Region')),
                "trend_line":       create_chart_image(plot_trend_line(analysis['cost_trend'])),
                "top_app_bar":      create_chart_image(plot_top_applications(analysis['top_application_costs'])),
                "rolling_avg":      create_chart_image(plot_rolling_average(analysis['cost_trend'])),
                "monthly_bars":     create_chart_image(plot_monthly_bars(analysis['monthly_service'])),
            }
        except Exception as exc:
            error = f"Could not analyse file — {exc}"
    else:
        error = "No file chosen. Please pick a CSV."

return render_template_string(PAGE, filename=filename, error=error, kpis=kpis, charts=charts)
```

Observed Result:

When a CSV file is uploaded, the system successfully analyzes the data, generates charts and cost metrics, and displays them on the dashboard. If no file or invalid data is provided, an appropriate error message is shown.

Purpose: Processes the uploaded CSV file by validating required columns, converting data types, and performing cost calculations such as total cost, average daily cost, and grouped analysis by department, region, service type, and application.

```
def analyze_costs(csv_file):
    df = pd.read_csv(csv_file)
    if 'Date' not in df.columns:
        raise ValueError('CSV must include a Date column.')
    df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
    if df['Date'].isna().all():
        raise ValueError('Could not parse any Date values.')
    if 'Cost_INR' not in df.columns:
        raise ValueError('CSV must include a Cost_INR column.')
    df['Cost_INR'] = pd.to_numeric(df['Cost_INR'], errors='coerce').fillna(0)
    df = df.dropna(subset=['Department', 'Service_Type', 'Application', 'Region', 'Date'])

    total_cost = int(df['Cost_INR'].sum())
    daily_cost = df.groupby('Date')['Cost_INR'].sum()
    avg_daily_cost = int(daily_cost.mean())

    department_cost       = df.groupby('Department')['Cost_INR'].sum().sort_values(ascending=False)
    region_cost           = df.groupby('Region')['Cost_INR'].sum().sort_values(ascending=False)
    service_costs         = df.groupby('Service_Type')['Cost_INR'].sum().sort_values(ascending=False)
    top_application_costs = df.groupby('Application')['Cost_INR'].sum().sort_values(ascending=False).head(8)

    # monthly cost by service type
    df['Month'] = df['Date'].dt.to_period('M')
    monthly_service = (
        df.groupby(['Month', 'Service_Type'])['Cost_INR']
        .sum()
        .unstack(fill_value=0)
        .sort_index()
```

```
return {
    'total_cost':            total_cost,
    'avg_daily_cost':        avg_daily_cost,
    'top_department_name':   department_cost.index[0],
    'top_department_cost':   int(department_cost.iloc[0]),
    'top_application_name':  top_application_costs.index[0],
    'top_application_cost':  int(top_application_costs.iloc[0]),
    'department_cost':       department_cost,
    'region_cost':           region_cost,
    'service_costs':         service_costs,
    'cost_trend':            daily_cost.sort_index(),
    'top_application_costs': top_application_costs,
    'monthly_service':       monthly_service,
}
```

Observed Result:

The system successfully cleans and analyzes the data, generating accurate cost summaries and structured insights that are used for visualization in the dashboard.

Purpose:

Creates and formats charts using Matplotlib and converts them into image format (base64) so they can be displayed directly on the web dashboard.

```
def _fig(w=6, h=4.2):
    fig, ax = plt.subplots(figsize=(w, h), dpi=110, facecolor=BG)
    fig.patch.set_facecolor(BG)
    ax.set_facecolor(SURFACE)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    ax.tick_params(colors=MUTED, labelsize=9)
    ax.grid(color=GRID, linewidth=0.6, alpha=0.7)
    return fig, ax

def create_chart_image(fig):
    buf = BytesIO()
    fig.savefig(buf, format='png', bbox_inches='tight', facecolor=BG)
    buf.seek(0)
    data = base64.b64encode(buf.read()).decode('utf-8')
    plt.close(fig)
    return data
```

Observed Result:

Charts are generated with proper styling and successfully converted into images, which are displayed clearly on the dashboard without saving files.

Purpose (Pie Chart): Creates a pie chart to visualize the percentage distribution of cloud cost across different service types.

```
def plot_service_pie(series):
    fig, ax = plt.subplots(figsize=(5, 4.2), dpi=110, facecolor=BG)
    fig.patch.set_facecolor(BG)
    ax.set_facecolor(BG)
    palette = ['#5b8af7','#34d399','#f59e0b','#a78bfa','#f87171','#22d3ee','#fb923c','#e879f9']
    wedges, _, autotexts = ax.pie(
        series.values,
        labels=series.index,
        autopct='%1.1f%%',
        startangle=140,
        colors=palette[:len(series)],
        textprops={'fontsize': 9, 'color': TEXT},
        wedgeprops={'linewidth': 1.2, 'edgecolor': BG},
        pctdistance=0.78,
    )
    for t in autotexts:
        t.set_color(TEXT)
        t.set_fontsize(8)
    ax.set_title('Service share', pad=14, color=TEXT, fontsize=11, fontweight='500')
    fig.tight_layout()
    return fig

def plot_bar(series, title, ylabel):
    fig, ax = _fig()
    colors = ['#5b8af7'] + ['#2d4a8a'] * (len(series) - 1)
    bars = ax.barh(series.index[::-1], series.values[::-1], color=colors[::-1],
                   height=0.62, linewidth=0)
    ax.set_title(title, pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Cost (₹)', color=MUTED, fontsize=9)
    ax.set_ylabel(ylabel, color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=TEXT)
    ax.grid(axis='x', color=GRID, linewidth=0.6)
    ax.grid(axis='y', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.tight_layout()
    return fig
```

Observed Result:

The chart clearly shows the cost share of each service with percentage values, making it easy to identify the highest cost-consuming services.

Purpose (Bar Chart):

Generates a horizontal bar chart to compare cloud costs across categories such as departments or regions.

```
def plot_service_pie(series):
    fig, ax = plt.subplots(figsize=(5, 4.2), dpi=110, facecolor=BG)
    fig.patch.set_facecolor(BG)
    ax.set_facecolor(BG)
    palette = ['#5b8af7','#34d399','#f59e0b','#a78bfa','#f87171','#22d3ee','#fb923c','#e879f9']
    wedges, _, autotexts = ax.pie(
        series.values,
        labels=series.index,
        autopct='%1.1f%%',
        startangle=140,
        colors=palette[:len(series)],
        textprops={'fontsize': 9, 'color': TEXT},
        wedgeprops={'linewidth': 1.2, 'edgecolor': BG},
        pctdistance=0.78,
    )
    for t in autotexts:
        t.set_color(TEXT)
        t.set_fontsize(8)
    ax.set_title('Service share', pad=14, color=TEXT, fontsize=11, fontweight='500')
    fig.tight_layout()
    return fig

def plot_bar(series, title, ylabel):
    fig, ax = _fig()
    colors = ['#5b8af7'] + ['#2d4a8a'] * (len(series) - 1)
    bars = ax.barh(series.index[::-1], series.values[::-1], color=colors[::-1],
                   height=0.62, linewidth=0)
    ax.set_title(title, pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Cost (₹)', color=MUTED, fontsize=9)
    ax.set_ylabel(ylabel, color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=TEXT)
    ax.grid(axis='x', color=GRID, linewidth=0.6)
    ax.grid(axis='y', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.tight_layout()
    return fig
```

Observed Result:

The chart displays cost comparisons clearly, helping users quickly identify which category has higher or lower cloud expenses.

Purpose (Trend Lines):

Visualizes daily cloud cost over time using a line chart to show trends and variations.

```
def plot_trend_line(series):
    fig, ax = _fig(w=7, h=4)
    ax.fill_between(series.index, series.values, alpha=0.12, color='#5b8af7')
    ax.plot(series.index, series.values, color='#5b8af7', linewidth=1.8, solid_capstyle='round')
    ax.scatter(series.index, series.values, color='#5b8af7', s=28, zorder=5,
               edgecolors=BG, linewidths=1.2)
    ax.set_title('Daily cost trend', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Date', color=MUTED, fontsize=9)
    ax.set_ylabel('Total Cost (₹)', color=MUTED, fontsize=9)
    ax.grid(color=GRID, linewidth=0.6)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.autofmt_xdate(rotation=30)
    fig.tight_layout()
    return fig

def plot_top_applications(series):
    fig, ax = _fig()
    colors = ['#34d399'] + ['#1a6649'] * (len(series) - 1)
    ax.barh(series.index[::-1], series.values[::-1], color=colors[::-1],
            height=0.62, linewidth=0)
    ax.set_title('Top applications', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Cost (₹)', color=MUTED, fontsize=9)
    ax.set_ylabel('Application', color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=TEXT)
    ax.grid(axis='x', color=GRID, linewidth=0.6)
    ax.grid(axis='y', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.tight_layout()
    return fig
```

Observed Result:

The chart clearly displays cost fluctuations, helping users identify patterns, peaks, and changes in daily cloud usage.

Purpose:

Displays the top cost-consuming applications using a horizontal bar chart.

```
def plot_trend_line(series):
    fig, ax = _fig(w=7, h=4)
    ax.fill_between(series.index, series.values, alpha=0.12, color='#5b8af7')
    ax.plot(series.index, series.values, color='#5b8af7', linewidth=1.8, solid_capstyle='round')
    ax.scatter(series.index, series.values, color='#5b8af7', s=28, zorder=5,
               edgecolors=BG, linewidths=1.2)
    ax.set_title('Daily cost trend', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Date', color=MUTED, fontsize=9)
    ax.set_ylabel('Total Cost (₹)', color=MUTED, fontsize=9)
    ax.grid(color=GRID, linewidth=0.6)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.autofmt_xdate(rotation=30)
    fig.tight_layout()
    return fig

def plot_top_applications(series):
    fig, ax = _fig()
    colors = ['#34d399'] + ['#1a6649'] * (len(series) - 1)
    ax.barh(series.index[::-1], series.values[::-1], color=colors[::-1],
            height=0.62, linewidth=0)
    ax.set_title('Top applications', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Cost (₹)', color=MUTED, fontsize=9)
    ax.set_ylabel('Application', color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=TEXT)
    ax.grid(axis='x', color=GRID, linewidth=0.6)
    ax.grid(axis='y', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.tight_layout()
    return fig
```

Observed Result:

The chart highlights the highest spending applications, making it easy to identify major cost contributors.

Purpose (Horizontal Bar Chart):

Generates a horizontal bar chart to display the top applications based on cloud cost usage.

```
def plot_top_applications(series):
    fig, ax = _fig()
    colors = ['#34d399'] + ['#1a6649'] * (len(series) - 1)
    ax.barh(series.index[::-1], series.values[::-1], color=colors[::-1],
            height=0.62, linewidth=0)
    ax.set_title('Top applications', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Cost (₹)', color=MUTED, fontsize=9)
    ax.set_ylabel('Application', color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=TEXT)
    ax.grid(axis='x', color=GRID, linewidth=0.6)
    ax.grid(axis='y', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.tight_layout()
    return fig
```

Observed Report:

The chart clearly highlights the highest cost-consuming applications, enabling easy identification of major cost contributors.

Purpose:

Calculates and visualizes the rolling average of daily cloud costs to provide a smoother trend over time.

```
def plot_rolling_average(series, window=7):
    fig, ax = _fig(w=12, h=4)
    rolling = series.rolling(window=window, min_periods=1).mean()
    # faint raw daily bars
    ax.fill_between(series.index, series.values, alpha=0.07, color='#5b8af7')
    ax.plot(series.index, series.values, color='#5b8af7', linewidth=0.8,
            alpha=0.30, solid_capstyle='round')
    # bold rolling average fill + smooth line
    ax.fill_between(series.index, rolling.values, alpha=0.18, color='#5b8af7')
    ax.plot(series.index, rolling.values, color='#5b8af7', linewidth=2.2,
            solid_capstyle='round')
    # dots only every ~14 days to avoid clutter
    step = max(1, len(series) // 26)
    sampled_idx = series.index[::step]
    sampled_val = rolling[::step]
    ax.scatter(sampled_idx, sampled_val, color='#5b8af7', s=34, zorder=5,
               edgecolors=BG, linewidths=1.4)
    ax.set_title(f'Daily cost trend · {window}-day rolling avg',
                 pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Date', color=MUTED, fontsize=9)
    ax.set_ylabel('Total Cost (₹)', color=MUTED, fontsize=9)
    ax.grid(color=GRID, linewidth=0.6)
    for spine in ax.spines.values():
        spine.set_color(GRID)
    fig.autofmt_xdate(rotation=30)
    fig.tight_layout()
    return fig
```

Observed Result:

The chart reduces fluctuations and clearly shows overall cost trends, making it easier to understand long-term patterns.

Purpose:

Generates grouped bar charts to display monthly cloud cost distribution across different service types such as Compute, Storage, Database, and Networking.

```python
def plot_monthly_bars(monthly_service):
    SERVICE_COLORS = {
        'Compute':    '#5b8af7',
        'Storage':    '#34d399',
        'Database':   '#a78bfa',
        'Networking': '#f59e0b',
    }
    services     = list(monthly_service.columns)
    n_months     = len(monthly_service)
    n_svc        = len(services)
    bar_w        = 0.7 / n_svc
    x            = np.arange(n_months)
    month_labels = [str(p) for p in monthly_service.index]

    fig, ax = _fig(w=max(12, n_months * 1.1), h=4.2)

    for i, svc in enumerate(services):
        offset = (i - n_svc / 2 + 0.5) * bar_w
        color  = SERVICE_COLORS.get(svc, '#94a3b8')
        ax.bar(
            x + offset,
            monthly_service[svc].values,
            width=bar_w * 0.88,
            color=color,
            linewidth=0,
            label=svc,
        )

    ax.set_xticks(x)
    ax.set_xticklabels(month_labels, rotation=30, ha='right', fontsize=9, color=MUTED)
    ax.set_title('Monthly cost by service type', pad=12, color=TEXT, fontsize=11, fontweight='500')
    ax.set_xlabel('Month', color=MUTED, fontsize=9)
    ax.set_ylabel('Total Cost (₹)', color=MUTED, fontsize=9)
    ax.xaxis.set_tick_params(labelcolor=MUTED)
    ax.yaxis.set_tick_params(labelcolor=MUTED)
    ax.grid(axis='y', color=GRID, linewidth=0.6)
    ax.grid(axis='x', visible=False)
    for spine in ax.spines.values():
        spine.set_color(GRID)
```

```
for spine in ax.spines.values():
    spine.set_color(GRID)
ax.legend(
    fontsize=9, framealpha=0, labelcolor=TEXT,
    loc='upper left', ncol=min(4, n_svc),
    handlelength=1.2, handleheight=0.9,
    handletextpad=0.5, columnspacing=1.4,
)
fig.tight_layout()
return fig
```

Observed Result:

The chart clearly shows monthly cost variations for each service, allowing users to compare trends and identify which services contribute most to cloud expenses over time.

3.3 Implementation

The implementation of the Cloud Cost Intelligence Platform for Smart City is carried out using Python, Flask, Pandas, and Matplotlib. The system is developed as a web-based application where users can upload cloud billing data in CSV format and obtain cost analysis along with visual insights.

Backend Implementation:

The backend is implemented using Flask in Python. It handles file upload, data processing, and result generation. The main function index() manages GET and POST requests. When a CSV file is uploaded, the system processes the file and generates key performance indicators (KPIs) and charts.

The analyze_costs() function reads the CSV file using Pandas and performs data cleaning and validation. It ensures required columns such as Date and Cost_INR are present.

The data is then grouped and analyzed to compute:

Total cloud cost

Average daily cost

Department-wise cost

Region-wise cost

Service-wise cost

Top application costs

Chart Generation Implementation:

Matplotlib is used to generate different types of charts.

Functions such as:

plot_service_pie() – shows cost distribution by service

plot_bar() – displays department and region cost comparison

plot_trend_line() – shows daily cost trend

plot_rolling_average() – smoothens cost trend for better understanding

plot_top_applications() – highlights highest cost applications

plot_monthly_bars() – compares monthly service usage

Each chart is converted into an image using base64 encoding through the create_chart_image() function. This allows charts to be directly embedded into the web page without saving files.

Frontend Implementation:

The frontend is developed using HTML, CSS, and JavaScript. It provides a simple interface where users can upload CSV files and view results.

JavaScript functions dynamically update the UI after processing.

For example:

File selection updates file name display

Charts and KPIs are rendered instantly after upload

No page reload is required

3.4 Testing

Testing was performed to ensure that the Cloud Cost Intelligence Platform for Smart City works correctly for different inputs and user actions. The system was tested using multiple CSV files and realtime scenarios.

Functional Testing:

- CSV file upload works correctly
- System reads and processes cloud billing data
- Required columns (Date, Cost_INR, etc.) are validated
- Total cost and average daily cost are calculated correctly
- Department, region, and service-wise grouping works properly
  Graphs (pie, bar, line) are generated without errors
- Dashboard displays all results correctly

Data Validation Testing:

- Valid CSV file with correct format
- Missing columns (Date / Cost_INR)
- Empty file
- Invalid or non-numeric values

Graph Testing:

Each graph was tested individually:

- Pie chart correctly shows service cost distribution
- Bar charts display department and region costs clearly
- Line chart shows daily cost trend accurately
- Rolling average smoothens fluctuations

- Monthly bar chart shows service trends per month

Performance Testing:

- Tested with small and medium-sized CSV files
- Checked response time for data processing and visualization

User Interface Testing:

- File upload interaction tested
- Dashboard display checked after upload
- Charts and KPIs visibility verified

3.5 Graphical User Interface (GUI) Layout

The Graphical User Interface (GUI) of the Cloud Cost Intelligence Platform for Smart City is designed as an interactive and user-friendly dashboard that enables users to easily upload, analyze, and visualize cloud cost data. The interface focuses on simplicity, clarity, and effective data presentation to ensure that even non-technical users can understand cloud usage insights.

The GUI is organized into multiple sections with a clear visual hierarchy to improve usability and navigation. At the top, the interface provides a file upload section, which allows users to select and upload a CSV file containing cloud billing data. Once a file is selected, the system displays the file name dynamically, confirming successful selection and improving user interaction.

After uploading the file, the dashboard presents Key Performance Indicators (KPIs) such as total cloud cost, average daily cost, top department, and top application cost. These metrics are displayed in a clear and structured format, allowing users to quickly understand the overall cost summary.

The central part of the interface consists of a charts and visualization section, which plays a major role in the system.

Various graphical representations are used, including:

- Pie Chart: Shows cost distribution across different service types
- Bar Charts: Compare costs across departments and regions
- Line Chart: Displays daily cost trends over time
- Rolling Average Chart: Smoothens fluctuations to show long-term patterns
- Monthly Bar Chart: Illustrates monthly cost variation by service

These charts are visually appealing and help users easily identify trends, patterns, and high-cost areas.

The interface is designed to update dynamically. Once the data is processed, the charts and metrics are displayed instantly without requiring a page reload. This provides a smooth and responsive user experience.

GUI Components Overview:

- File Upload Section: Allows users to select and upload CSV files.
- Dashboard Metrics Section: Displays key cost insights (KPIs).
- Visualization Section: Shows multiple charts for analysis.
- Result Display Area: Presents processed data clearly.

UI Design Principles Followed:

The GUI design follows standard user interface principles to ensure better usability:

- Clean Layout: Organized sections with proper spacing.
- Visual Hierarchy: Important data (KPIs, charts) highlighted clearly.
- Consistency: Uniform colors, fonts, and design across the interface.
- Consistency: Uniform colors, fonts, and design across the interface.
- Readability: Clear text, labels, and chart titles.
- User-Friendly Interaction: Minimal steps required to perform actions.

3.6 Customer Testing

Customer testing was conducted to evaluate the usability, functionality, and overall effectiveness of the Cloud Cost Intelligence Platform for Smart City. A small group of users (students/peers) were asked to interact with the system by uploading CSV files and analyzing cloud cost data through the dashboard. The main objective of this testing was to understand how easily users could operate the system, interpret the results, and identify any usability issues. Users were given sample cloud billing datasets and asked to perform tasks such as uploading files, viewing charts, and understanding cost insights.

Testing Activities:

The following activities were performed during customer testing:

- Uploading CSV files with cloud billing data.
- Viewing dashboard metrics such as total cost and average daily cost.
- Interpreting visual charts (pie, bar, and trend graphs).

- Identifying high-cost services, departments, and applications.
- Checking system response to invalid or incorrect inputs.

Observed Testing:

The system was tested with users to evaluate usability and performance, and the following observations were made:

- Users were able to upload files without difficulty.
- Dashboard results were displayed clearly and quickly.
- Charts helped users easily understand cost distribution and trends.
- The interface was considered simple and easy to use.
- Users completed tasks without needing technical knowledge.

User Feedback:

Some common feedback received from users includes:

- The dashboard is clean and easy to understand.
- Graphs are helpful for visualizing cloud cost data.
- The system provides quick results after file upload.
- More interactive features like filters or search could improve usability.
- Additional details or explanations for charts could enhance understanding

Interpretation:

The testing results indicate that the system is user-friendly, functional, and effective for cloud cost analysis. The GUI design and visualizations help users quickly understand complex data. The platform successfully meets its objective of providing a simple and interactive solution for analyzing cloud expenses in a smart city context.

3.7 Evaluation

The project successfully achieves its primary objective of developing a Cloud Cost Intelligence Platform for Smart City that analyzes and visualizes cloud billing data in an efficient and user-friendly manner.

The system is capable of processing CSV-based cloud cost data, performing accurate analysis using Pandas, and presenting the results through an interactive web dashboard.
The integration of Flask (backend), Pandas (data processing), and Matplotlib (visualization) ensures a smooth workflow from data input to output. The platform effectively transforms raw cloud billing data into meaningful insights such as cost distribution, trends, and key performance indicators. The use of graphical representations like pie charts, bar graphs, and trend lines enhances understanding and supports better decision-making.

Strengths:

• Clear and modular architecture with separation of frontend, backend, and data processing
• Efficient handling and analysis of cloud billing data using Pandas
• Multiple visualizations that improve data understanding
• Lightweight and easy-to-deploy system without complex dependencies
• Good usability with a simple and clean dashboard interface
• Demonstrates practical application of data analytics in cloud cost management
• Easy to explain and present during viva and academic evaluation

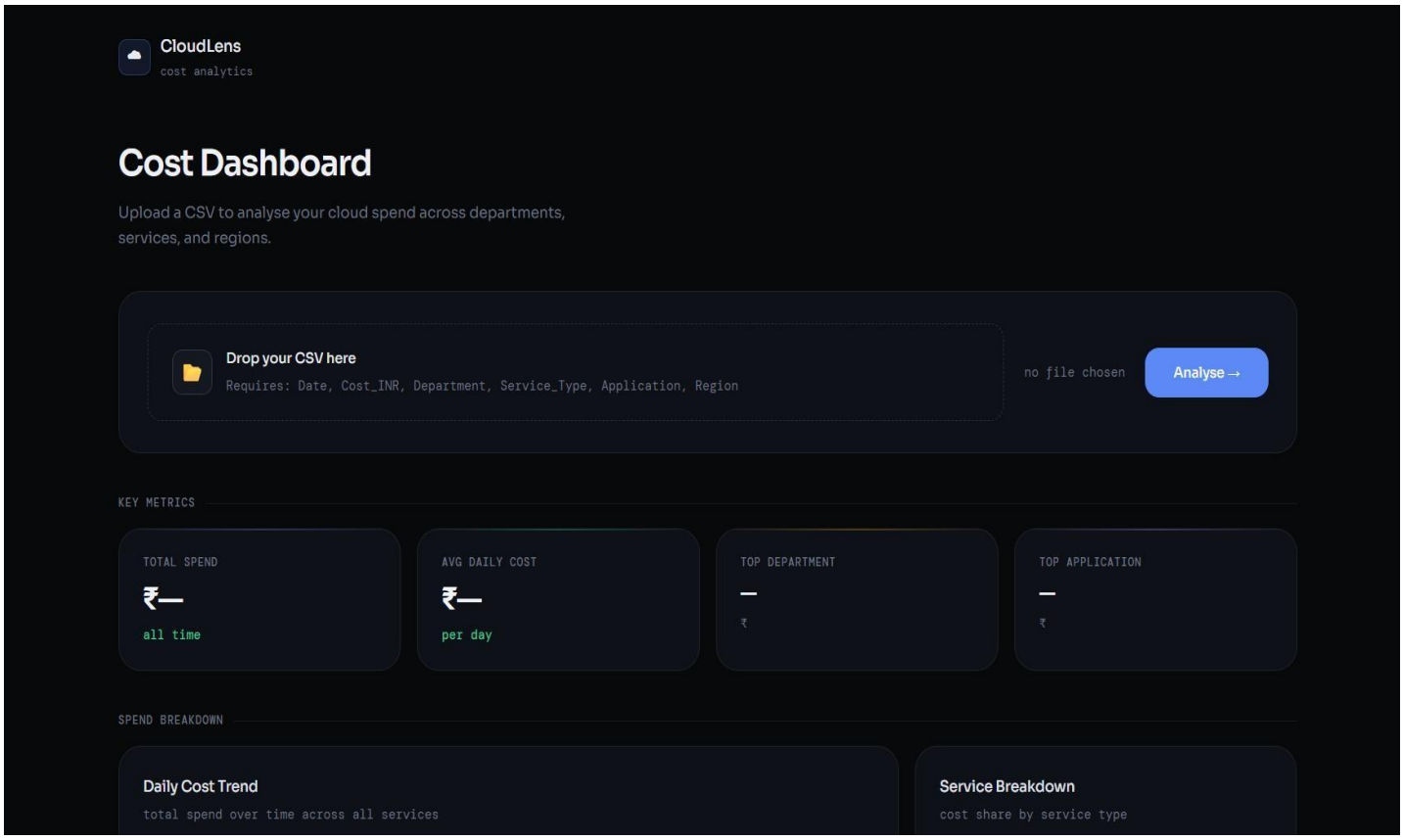
Limitations:

• No database integration; data is processed only in-memory.
• Lack of real-time cloud data integration.
• Limited advanced analytics (no predictive or AI-based insights).
• Basic user interface without advanced interactivity (filters, drill-down, etc.).
• No user authentication or role-based access control.
• Security features are minimal for production-level deployment.
• Testing is mainly manual; no automated testing framework implemented.

4.Snapshots of the Project

i) Home / Dashboard View

Displays the main interface of the system with file upload option and dashboard layout. It provides an overview of how users interact with the application.



SPEND BREAKDOWN

Daily Cost Trend
total spend over time across all services

upload a CSV to render

Service Breakdown
cost share by service type

upload a CSV to render

By Department
ranked by total spend

upload a CSV to render

Regional Distribution
cost by cloud region

upload a CSV to render

Top Applications
highest spend, top 8

upload a CSV to render

Rolling Average Trend
daily spend · 7-day rolling average overlay

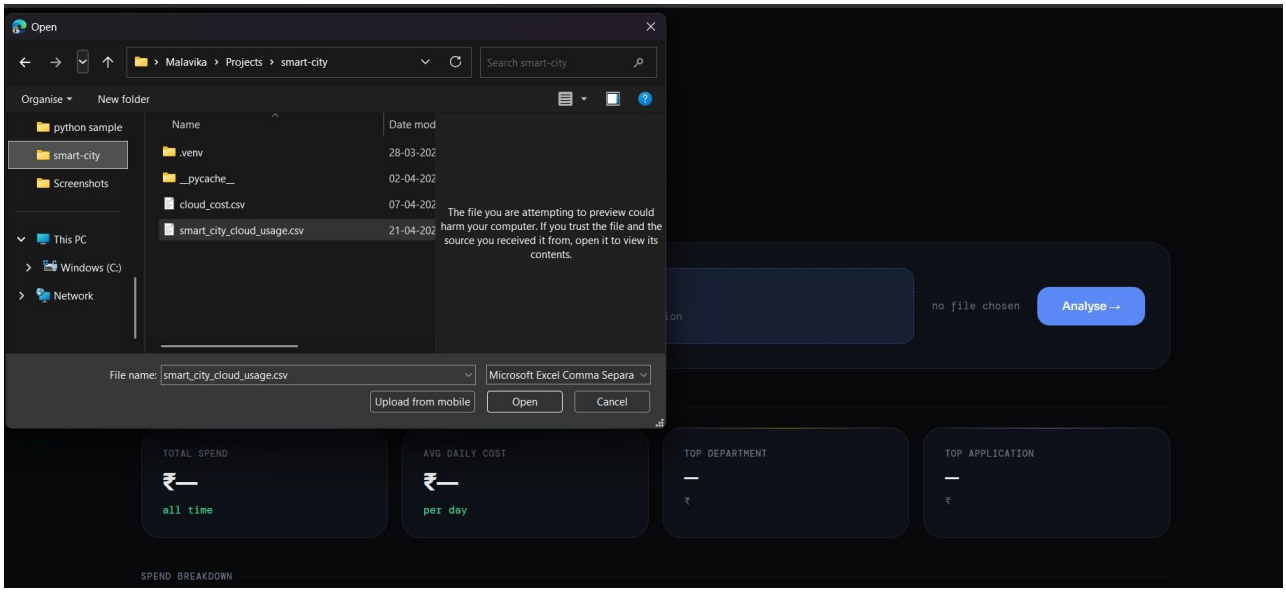
upload a CSV to render

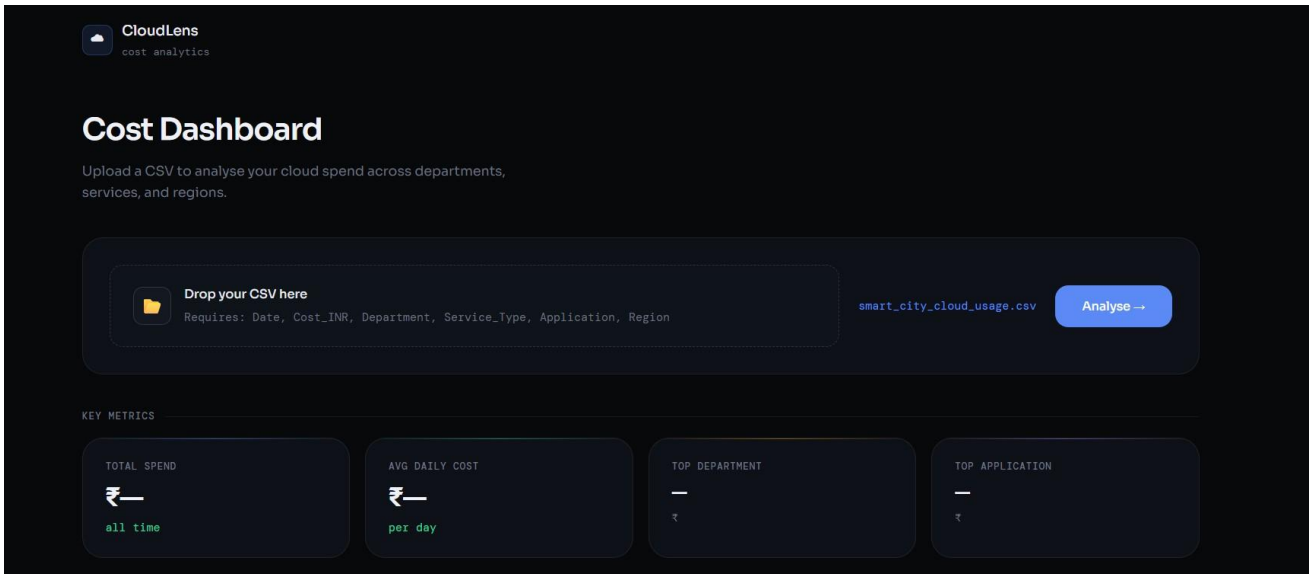
Monthly Cost by Service
grouped by service type

upload a CSV to render

ii) File Upload Section

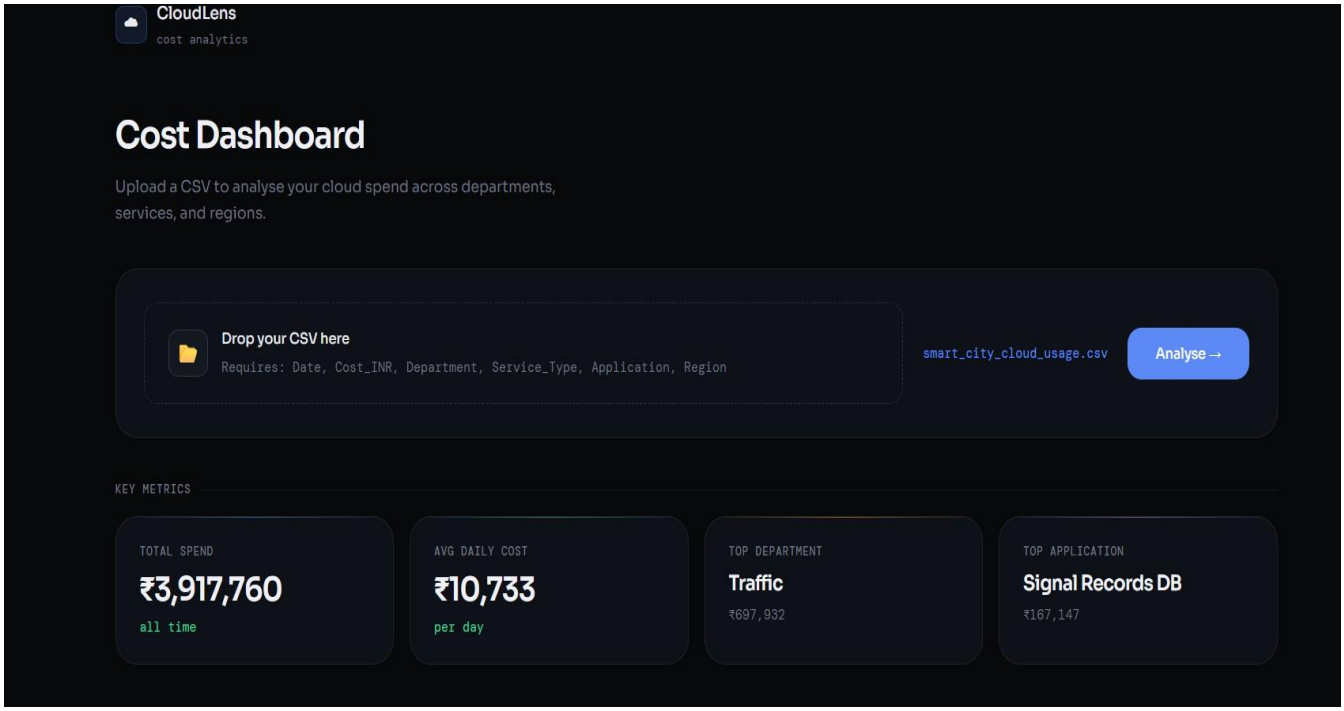
Shows the CSV file upload section where users can select and upload cloud data. The file name is displayed, and users can analyze the data easily.





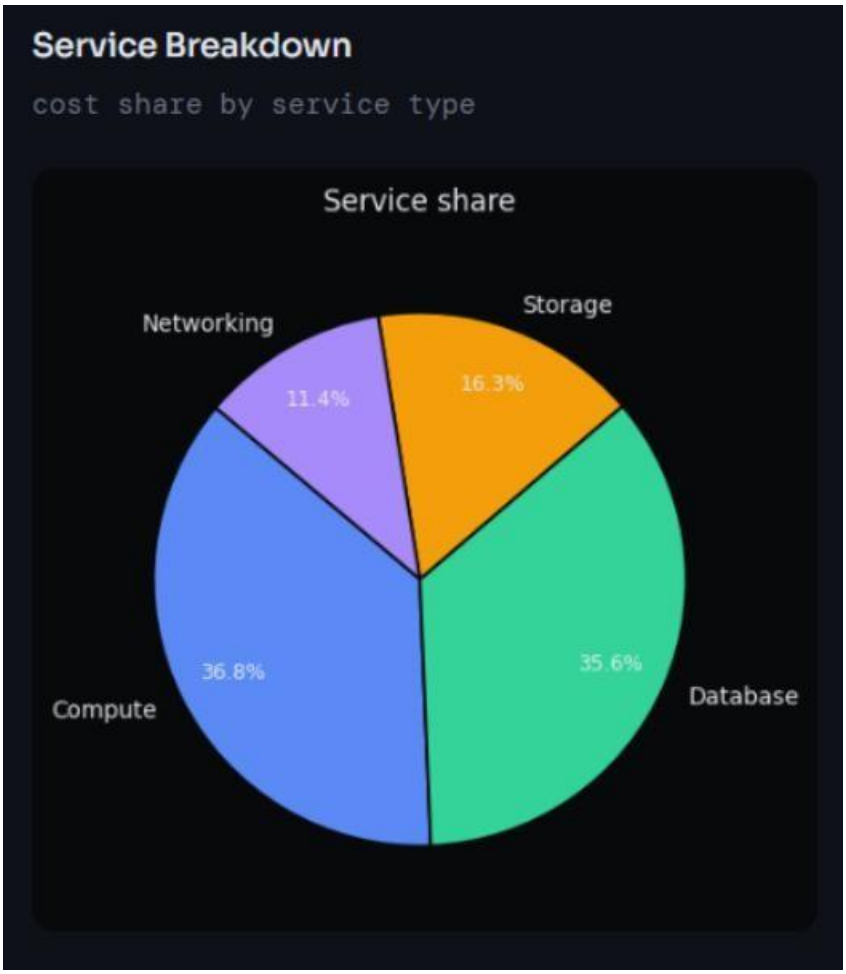
iii) KPI Dashboard

Displays key performance indicators such as total cost, average daily cost, top department, and top application.



iv) Service Distribution (Pie Chart)

Shows cost distribution across different cloud services, helping users identify major cost contributors.



v) Cost Trend (Line Chart)

Shows daily cloud cost trends over time, helping users understand usage patterns.

Vi) Department / Region Analysis (Bar Charts)

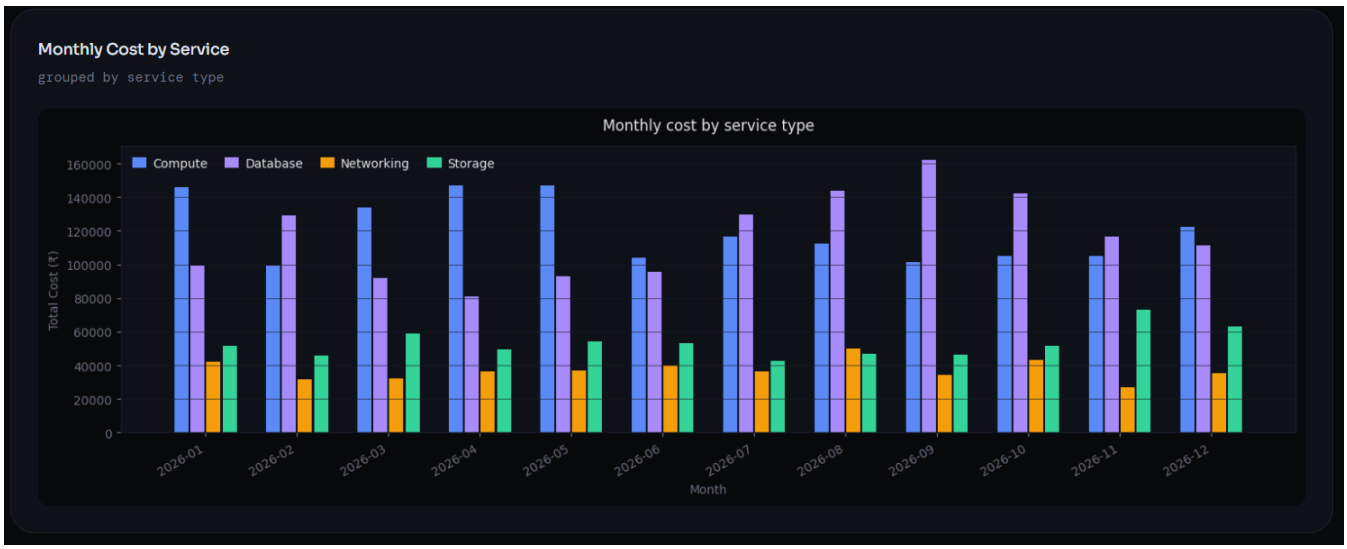
Displays cost comparison across departments and regions using bar graphs.

vii) Rolling Average Chart

Displays smoothed cost trends for better long-term analysis.

viii) Monthly Cost Analysis

Shows monthly cost variation across different service types.



5.Conclusion

The Cloud Cost Intelligence Platform for Smart City successfully demonstrates how cloud billing data can be analyzed and visualized using simple and effective tools. The system processes CSV-based cloud cost data and converts it into meaningful insights such as total cost, usage trends, and service-wise distribution.

By integrating Python, Flask, Pandas, and Matplotlib, the project provides a complete workflow from data input to visualization. The dashboard-based interface makes it easy for users to understand complex cloud cost data through charts and key performance indicators.

The project highlights the importance of data analysis in cloud cost management and shows how visualization can support better decision-making. Overall, the system is simple, efficient, and serves as a strong foundation for developing more advanced cloud cost monitoring solutions in the future.

6. Further Development or Research

Recommended Future Scope:

• Integrate a relational database (MySQL/PostgreSQL) for efficient data storage and scalability
• Add user authentication and role-based access (admin/user)
• Enable real-time cloud data integration using APIs (AWS, Azure, GCP)
• Implement cost alert and notification system for budget control
• Add advanced dashboard features like filters, drill-down analysis, and comparisons
• Provide report export options (PDF/Excel) for analysis sharing
• Develop an admin panel for monitoring and managing cloud usage data
• Introduce automated testing (unit, integration, UI testing)
• Deploy the system on cloud platforms with CI/CD pipeline
• Improve UI/UX with more interactive and responsive design
• Add multi-cloud cost comparison features
• Implement caching techniques for better performance

Research Directions:

• Machine learning models for cloud cost prediction and forecasting
• Anomaly detection techniques for identifying unusual cloud spending
• AI-based cost optimization and recommendation systems
• Real-time data analytics for continuous cloud monitoring
• Comparative analysis of different cloud providers (AWS, Azure, GCP)
• Study of scalable architectures for large cloud cost platforms
• Research on energy-efficient and sustainable cloud usage in smart cities
• Intelligent budgeting systems for automated cost control

7.References

- Python Software Foundation. Python Documentation. https://docs.python.org/3/
- Flask Documentation. Flask Web Framework. https://flask.palletsprojects.com/
- Pandas Documentation. Data Analysis Library. https://pandas.pydata.org/docs/
- Matplotlib Documentation. Visualization Library https://matplotlib.org/stable/index.html
- NumPy Documentation. Numerical Computing in Python. https://numpy.org/doc/
- MDN Web Docs. HTML, CSS, JavaScript. https://developer.mozilla.org/
- Amazon Web Services (AWS). Cloud Billing and Cost Management. https://aws.amazon.com/aws-cost-management/
- Microsoft Azure. Cost Management and Billing. https://azure.microsoft.com/en-in/services/cost-management/
- Google Cloud Platform. Cloud Billing Documentation. https://cloud.google.com/billing/docs
- W3C. Web Standards Documentation. https://www.w3.org/

8. Appendix

Complete Source File List:

- fileanalyzer.py(Main Flask application)
- templates/fileanalyzer.html (Frontend dashboard UI)
- Sample CSV files (Cloud billing data for testing)
- Jupyter Notebook (optional for analysis/testing)

API / System Workflow:

- File upload request (CSV input handling)
- Data processing using Pandas
- Chart generation using Matplotlib
- Dashboard rendering with KPIs and graphs

Test Case Checklist and Outcomes:

- Valid CSV upload → Successfully processed
- Missing columns → Error message displayed
- Invalid data → Handled correctly
- Charts generation → Displayed accurately
- Dashboard update → Works without errors

9. Weekly Report

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 01

For week commencing 02 March-08 March 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 01 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To Understand the fundamentals of python programming
- To learn about variables, datatypes and operators
- To study control structures (if-else, loops)
- To practice basic input/output operations
- To setup Python environment and Jupyter Notebook

Progress/Achievements for the week:

- Successfully learned Python syntax and basic concepts
- Implemented programs using conditional statements and loops
- Created functions and understood code reusability
- Practiced data structures like lists and dictionaries
- Completed file handling programs (read & write operations)
- Explored basic usage of libraries like NumPy/Pandas
- Started mini project and completed initial setup

Future Work Plans:

- To practice more Python programs
- To improve understanding of loops and functions
- To learn basic concepts of Pandas
- To work with simple CSV files
- To understand data analysis step by step
- To improve coding skills through practice
- To start small tasks related to the project

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

Innovation Centre for Education

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 02

For week commencing 09 March-15 March 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 02 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To learn basics of Pandas library
- To understand working with CSV and Excel files
- To load and explore cloud billing dataset
- To perform data cleaning and preprocessing
- To analyze data using grouping (Service Type / Month)
- To understand basic data visualization techniques

Progress/Achievements for the week:

- Practiced Python programs for better understanding
- Improved knowledge of loops and functions
- Learned introduction to Pandas library
- Successfully read CSV file using Pandas
- Understood basic dataset structure (rows and columns)
- Performed simple data operations

Future Work Plans:

- To learn more Pandas functions
- To perform data analysis step by step
- To start grouping and filtering data
- To create basic charts for visualization
- To gradually move into project implementation

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 03

For week commencing 16 March-22March 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 03 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To learn more about data analysis concepts
- To understand how cloud billing data works
- To study basics of Pandas in detail
- To learn how to read CSV and Excel files
- To understand grouping and filtering of data
- To explore simple data visualization concepts

Progress/Achievements for the week:

- Gained better understanding of data analysis concepts
- Studied structure of cloud billing dataset
- Practiced reading CSV and Excel files using Pandas
- Learned grouping and filtering of data
- Explored basic ideas of data visualization
- Improved overall understanding of project requirements

Future Work Plans:

- To start creating simple charts and visualizations
- To apply data analysis on real dataset
- To improve Pandas skills
- To begin small implementation tasks
- To move deeper into project development

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 04

For week commencing 23 March-29March 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 04 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To start implementation using Python and Pandas
- To load and analyze cloud billing dataset
- To apply data cleaning techniques
- To perform grouping based on service type and month
- To begin creating simple charts
- To understand cost distribution patterns

Progress/Achievements for the week:

- Started working in Jupyter Notebook
- Successfully loaded cloud billing dataset (CSV file)
- Performed basic data cleaning and preprocessing
- Applied grouping using Pandas (service type and month)
- Analyzed cost distribution from dataset
- Created initial simple charts for visualization

Future Work Plans:

- To improve visualizations (pie charts, graphs)
- To develop cost calculator logic
- To integrate Excel data handling
- To enhance accuracy of analysis
- To continue project development

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 05

For week commencing 30 March-05 April 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 05 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To design a simple web page using HTML and CSS
- To create a user interface for file upload
- To integrate file upload option for CSV files
- To connect frontend with Python backend
- To test uploading and reading CSV files
- To improve user interaction and design

Progress/Achievements for the week:

- Successfully created a web page using VS Code
- Designed basic UI for the project interface
- Implemented file upload button for CSV files
- Created sample CSV file for testing
- Connected Python script to read uploaded file
- Verified file upload and data reading functionality

Future Work Plans:

- To generate analysis results from CSV file
- To display output (charts/summary) on web page
- To improve UI design and responsiveness
- To add more features for better user experience

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

Innovation Centre for Education

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 06

For week commencing 06 April-12 April 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 06 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To start implementation of the project
- To load and work with cloud billing dataset
- To apply data analysis using Pandas
- To perform grouping based on service type and month
- To begin creating simple visualizations
- To understand cost distribution pattern

Progress/Achievements for the week:

- Started implementation using Jupyter Notebook
- Successfully loaded and processed cloud billing dataset
- Applied grouping techniques using Pandas
- Analyzed cost based on service type and monthly usage
- Created basic visualizations for understanding data
- Gained practical experience in project implementation

Future Work Plans:

- To improve chart design and visualization quality
- To add more filters (date, service type)
- To enhance accuracy and performance
- To finalize cost calculator features

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**

WEEKLY PROJECT PROGRESS REPORT (WPPR)– 07

For week commencing 13 April-19 April 2026

Course: BCA-Artificial Intelligence, Cloud Computing and DevOPS

Student Name: Malavika Rajesh Register Number: 23BCAICD050

WPPR: 07 Internal Guide's Name: Ms. Niharika

MAJOR PROJECT Title:

CLOUD COST INTELLIGENCE PLATFORM FOR SMART CITY.

Targets set for the week:

- To finalize all project features
- To improve user interface and overall design
- To test complete system functionality
- To fix errors and optimize performance
- To ensure proper working of cost calculator

Progress/Achievements for the week:

- Successfully completed project implementation
- Improved UI design and user experience
- Tested file upload, data analysis, and output display
- Fixed minor errors and improved performance
- Verified accuracy of cost calculations
- Completed major parts of documentation

Future Work Plans:

- The project can be deployed as a real-time web application for practical use
- Integration with real-time cloud APIs can enhance accuracy of cost analysis
- Advanced filters and customization features can be added for better user experience
- Data visualization can be improved using more interactive charts
- System performance and scalability can be enhanced for large-scale applications
- Security features such as user authentication can be implemented

Implementation shown: ☐ Yes NO ☐

Remarks by the Internal Guide:

Signature of the student **Signature of the Internal Guide**